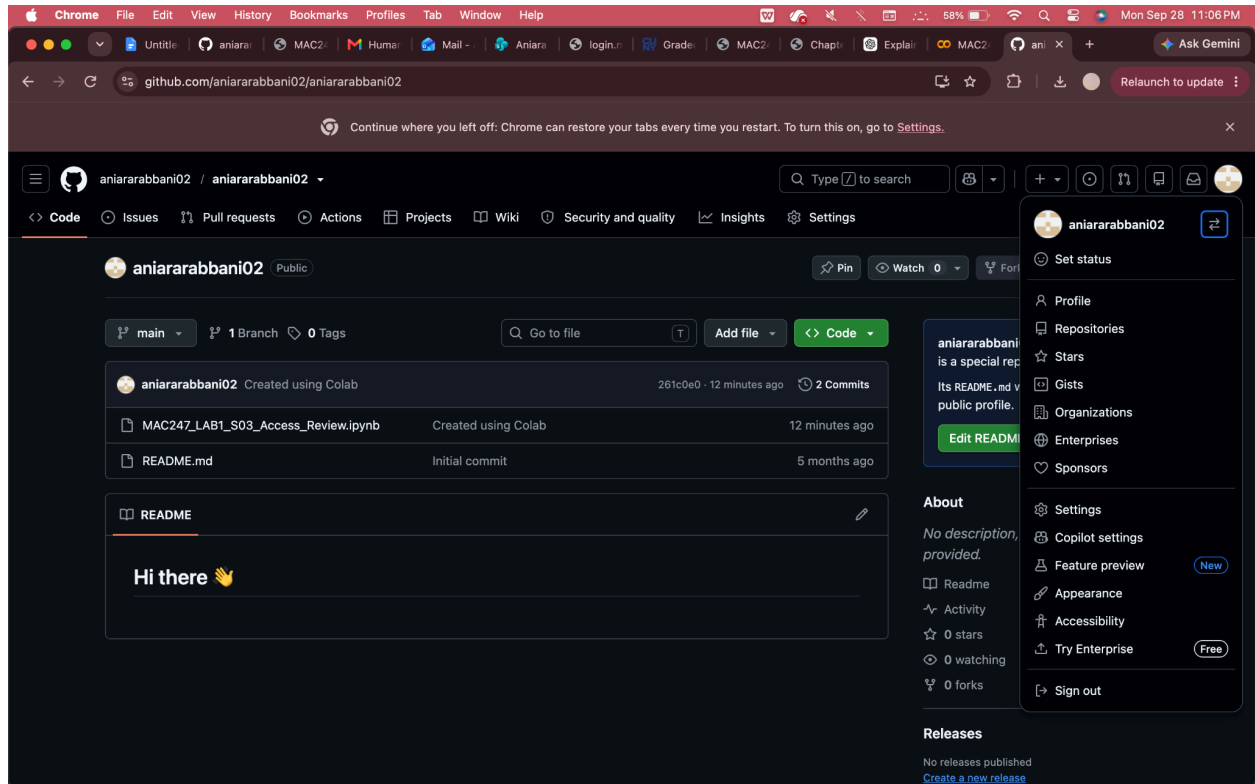


PART 1 CLOSING ANSWERS:

- 1) DAC fits a two-person startup because resource owners can directly decide who receives access. MAC fits a fifty-thousand-person defense contractor because access is centrally enforced through security classifications and user clearances that individual owners cannot override. ABAC fits an internal data lake where access depends on the row because access decisions can use attributes of the user and the specific data being accessed.
- 2) MAC denied finance_lead because writing from the restricted clearance level to the internal audit.log violates the Bell-LaPadula no write down rule. This prevents restricted or sensitive information from being placed in a lower-classified file where less-cleared users may be able to access it.
- 3) The 2:00 AM request failed the within work hours clause, the personal-laptop request failed the managed device clause, and the 7:00 PM request failed the within work hours clause. None of DAC, MAC, or RBAC in this model could express those time-of-day or device-state conditions.
- 4) Part 2 should report that mac247_backup is over-entitled because it has the unnecessary group:mac247aud entitlement, which also creates a separation-of-duties conflict between backup and auditing duties. Part 2 should also report that mac247_analyst is a stale account because it has remained active without being maintained or used for an excessive period.

Screenshots:

Repository setup:



Model check results screenshot:

```
[7] ✓ Os
CASES = [
    ('DAC analyst_a writes ledger.csv', dac('analyst_a', 'ledger.csv', 'write'), False),
    ('DAC auditor_a reads backup.tar', dac('auditor_a', 'backup.tar', 'read'), True),
    ('MAC analyst_a reads ledger.csv', mac('analyst_a', 'ledger.csv', 'read'), False),
    ('MAC finance_lead writes audit.log', mac('finance_lead', 'audit.log', 'write'), False),
    ('RBAC auditor_a reads audit.log', rbac('auditor_a', 'audit.log', 'read'), True),
    ('RBAC analyst_a writes payroll.db', rbac('analyst_a', 'payroll.db', 'write'), False),
]

bad = 0
for name, got, want in CASES:
    ok = (got == want)
    bad += (not ok)
    print(f'{"PASS" if ok else "FAIL"} {name:36s} decision={str(got):5s} expected={want}')
assert bad == 0, 'a decision function disagrees with the model it claims to implement'
print('\n CHECK PASS: all six decisions match the model')
print(' token:', TOKEN)

... PASS DAC analyst_a writes ledger.csv decision=False expected=False
PASS DAC auditor_a reads backup.tar decision=True expected=True
PASS MAC analyst_a reads ledger.csv decision=False expected=False
PASS MAC finance_lead writes audit.log decision=False expected=False
PASS RBAC auditor_a reads audit.log decision=True expected=True
PASS RBAC analyst_a writes payroll.db decision=False expected=False

CHECK PASS: all six decisions match the model
token: 53f35e2938153e33
```

Assignment card:

```
[10] ✓ Os
excess_role = candidates[h % len(candidates)]
choices = conflicting_additions(excess_role)
excess_entitlement = choices[(h // 7) % len(choices)]
stale_role = roles[(h // 31) % len(roles)]

CARD = {
    'fault_1_over_entitled_account': ROSTER[excess_role],
    'fault_1_extra_entitlement': excess_entitlement,
    'fault_2_stale_account': ROSTER[stale_role],
}
for k, v in CARD.items():
    print(f'{k:32s} {v}')

assert excess_entitlement not in ROLE_REQUIRED[excess_role], 'not actually an excess'
assert excess_entitlement in conflicting_additions(excess_role), 'not actually a conflict'
print('\n CHECK PASS: the extra entitlement is outside the role that receives it and '
      'collides with a duty it already holds')
print(' token:', TOKEN)

... fault_1_over_entitled_account mac247_backup
fault_1_extra_entitlement group:mac247aud
fault_2_stale_account mac247_analyst

CHECK PASS: the extra entitlement is outside the role that receives it and collides with a duty it already holds
token: 53f35e2938153e33
```

PART 2:

1. Why are the over-entitlement and separation-of-duties break two different findings?

The over-entitlement and separation-of-duties break are two findings because they measure two different problems. The over-entitlement means mac247_backup has group:mac247aud, an access right that its backup role does not require, so fixing it is relatively simple: remove that unnecessary group membership. The separation-of-duties finding is about the combination of group:mac247bcp and group:mac247aud, which allows the same account to both hold the backups and attest that the backups are intact; if both permissions were actually needed for business reasons, fixing that conflict could require separating the duties between different accounts or changing the process. In this lab, removing the unnecessary audit group happens to fix both findings, but they represent different risks.

2. What makes the over-entitlement visible?

The finding becomes visible when identities.csv is compared with entitlements.csv and the role requirements defined in the model. identities.csv only shows information such as the username, primary group, shell, lock status, and last password change, while entitlements.csv lists every entitlement the account actually holds. An account-by-account review of the identity row alone would miss the problem because mac247_backup looks normal until its complete entitlement set is compared with what the backup role is actually supposed to have.

3. What identity-life-cycle stage failed for the stale account?

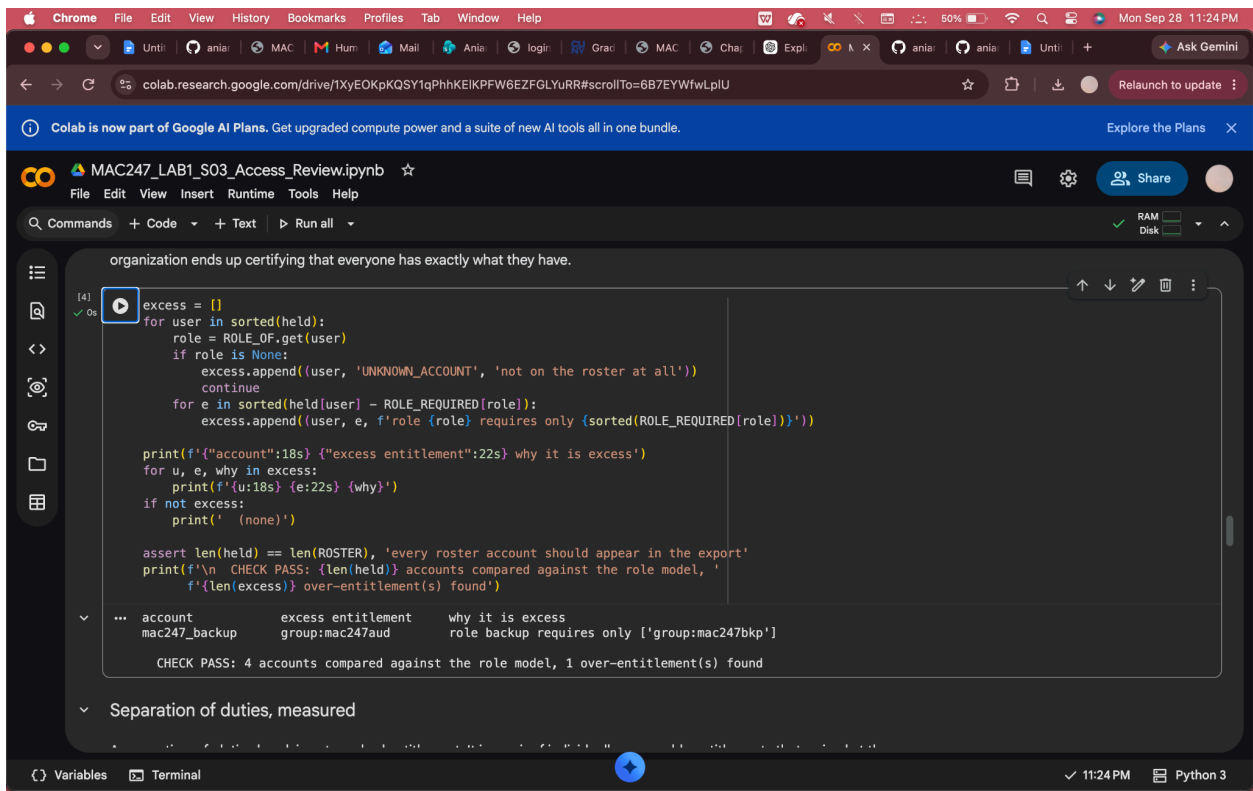
The entitlement review/maintenance stage failed because the account remained active and unlocked even though nobody had touched it for more than a year. The stale mac247_analyst account still owned its analyst entitlement, group:mac247ana, meaning unused access remained available on the system instead of being reviewed and removed or disabled.

4. NIST SP 800-53 Rev. 5 control

NIST SP 800-53 Rev. 5 AC-6, Least Privilege, requires access to be limited to only the permissions necessary for users or processes to perform their assigned organizational tasks. The command `sudo gpasswd -d mac247_backup mac247aud` provides evidence of implementing AC-6 because it removes the unnecessary audit-group entitlement from the backup account, leaving it with only the access required for its assigned role.

Screenshots:

1) Access review findings



organization ends up certifying that everyone has exactly what they have.

```
[4] ✓ Os
excess = []
for user in sorted(held):
    role = ROLE_OF.get(user)
    if role is None:
        excess.append((user, 'UNKNOWN_ACCOUNT', 'not on the roster at all'))
        continue
    for e in sorted(held[user] - ROLE_REQUIRED[role]):
        excess.append((user, e, f'role {role} requires only {sorted(ROLE_REQUIRED[role])}'))

print(f'{"account":18s} {"excess entitlement":22s} why it is excess')
for u, e, why in excess:
    print(f'{"u":18s} {"e":22s} {"why}'))
if not excess:
    print(' (none)')

assert len(held) == len(ROSTER), 'every roster account should appear in the export'
print(f'\n CHECK PASS: {len(held)} accounts compared against the role model, '
      f'{len(excess)} over-entitlement(s) found')
```

account	excess entitlement	why it is excess
mac247_backup	group:mac247aud	role backup requires only ['group:mac247bcp']

CHECK PASS: 4 accounts compared against the role model, 1 over-entitlement(s) found

Separation of duties, measured

2) Separation of duties

Colab is now part of Google AI Plans. Get upgraded compute power and a suite of new AI tools all in one bundle. [Explore the Plans](#)

MAC247_LAB1_S03_Access_Review.ipynb

File Edit View Insert Runtime Tools Help

Commands + Code + Text Run all

RAM Disk

Separation of duties, measured

A separation-of-duties break is not one bad entitlement. It is a pair of individually reasonable entitlements that arrived at the same person, usually months apart, usually because somebody changed job and kept the old access. Nothing about either entitlement looks wrong on its own, which is exactly why this needs a population-level check rather than an eye.

```
[5] sod = []
for user in sorted(held):
    for side_a, side_b, why in SOD_CONFLICTS:
        if side_a <= held[user] and side_b <= held[user]:
            sod.append((user, sorted(side_a)[0], sorted(side_b)[0], why))

for u, a, b, why in sod:
    print(f'CONFLICT {u}')
    print(f'    holds {a} and {b}')
    print(f'    which together allow {why}\n')
if not sod:
    print('no separation-of-duties conflicts in this export')

checked = len(held) * len(SOD_CONFLICTS)
print(f' CHECK PASS: {checked} account-and-conflict pairs tested, {len(sod)} break(s) found')
```

CONFLICT mac247_backup
holds group:mac247bcp and group:mac247aud
which together allow holding the backups and attesting that the backups are intact

CHECK PASS: 12 account-and-conflict pairs tested, 1 break(s) found

Variables Terminal 11:24 PM Python 3

3)

Colab is now part of Google AI Plans. Get upgraded compute power and a suite of new AI tools all in one bundle. [Explore the Plans](#)

MAC247_LAB1_S03_Access_Review.ipynb

File Edit View Insert Runtime Tools Help

Commands + Code + Text Run all

RAM Disk

Section 4. Did the review find what was planted?

This is the check that separates a detector from a plausible-looking report. You knew before the export was written what was wrong with it. If either line below does not read found, the detector is not detecting, and finding that out is worth more to you than a clean run.

```
[7] excess_hits = [(u, e) for u, e, _ in excess
                  if u == CARD['fault_1_over_entitled_account']
                  and e == CARD['fault_1_extra_entitlement']]
sod_hits = [u for u, _, _ in sod if u == CARD['fault_1_over_entitled_account']]
stale_hits = [u for u, _, _ in stale if u == CARD['fault_2_stale_account']]

print(f'fault 1, {CARD["fault_1_extra_entitlement"]} on '
      f'{CARD["fault_1_over_entitled_account"]}:18s} -> '
      f'{"found" if excess_hits else "NOT FOUND"} as an over-entitlement, '
      f'{"found" if sod_hits else "NOT FOUND"} as a separation-of-duties break')
print(f'fault 2, stale account {CARD["fault_2_stale_account"]}:18s} -> '
      f'{"found" if stale_hits else "NOT FOUND"}')

assert excess_hits and sod_hits and stale_hits, 'the review missed a fault it was told about'
print('\n CHECK PASS: both planted faults were recovered from the exported files')
print(' token:', TOKEN)
```

fault 1, group:mac247aud on mac247_backup -> found as an over-entitlement, found as a separation-of-duties break
fault 2, stale account mac247_analyst -> found

CHECK PASS: both planted faults were recovered from the exported files
token: f6a36e4a50b017f5

Variables Terminal 11:24 PM Python 3

```
plan = []
for u, e, _ in excess:
    kind = 'group' if e.startswith('group:') else 'sudo'
    cmd, control = REMEDIATION[kind]
    plan.append((cmd.format(user=u, name=e.split(':', 1)[1]), control,
                f'{u} holds {e}, which its role does not require'))

for u, age, _ in stale:
    cmd, control = REMEDIATION['stale']
    plan.append((cmd.format(user=u), control,
                f'{u} has not changed its password in {age} days'))

print(f'# MAC 247 Lab 1 remediation plan, token', TOKEN)
for cmd, control, why in plan:
    print(f'\n# {why}\n# control: {control}\n# cmd:')

assert plan, 'no findings, so no plan: check that the export was written before this ran'
print(f'\n CHECK PASS: {len(plan)} command(s) generated, each tied to one finding '
      'and one control')

... # MAC 247 Lab 1 remediation plan, token f6a36e4a50b017f5

# mac247_backup holds group:mac247aud, which its role does not require
# control: AC-6 Least Privilege
sudo gpasswd -d mac247_backup mac247aud

# mac247_analyst has not changed its password in 430 days
# control: AC-2(3) Account Management, Disable Accounts
sudo usermod -L mac247_analyst && sudo chage -E 0 mac247_analyst

CHECK PASS: 2 command(s) generated, each tied to one finding and one control
```

4)

REFLECTION:

I learned from this lab how access control problems can exist even when an account looks normal at first. Before this, I might have assumed that checking a username, role, or account status was enough to decide whether access was appropriate. However, the lab shows that an access review requires comparing the permissions an account has with the permissions its role is supposed to have. For example, the backup account had an unnecessary audit-group entitlement, which created both an over-entitlement issue and a separation-of-duties conflict. This helped me understand that one extra permission can introduce more than one type of security risk. I also learned that stale accounts are dangerous because unused access can remain active if it is not reviewed and removed. Overall, this lab showed me why least privilege, access reviews, and separation of duties are important parts of managing secure systems.